
**<Large-Scale Design & Analysis of Neural
Networks>
System Design Document
Version <1.0>
11/21/2024**

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Clay Pate

Customer(s): M. Ilhan Akbas

John M. Thompson

Project team members:

Daniel Machado

Simon Cole

Muhammed Sharif Adepetu

Leah Powell

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
0.1	10/15/2024	Leah	Document Creation
0.2	10/29/2024	Muhammed-Sharif	Edits
0.3	11/4/2024	Daniel Machado	1.1, 1.2., 1.2.2, 1.2.3 1.4, 1.5
0.4	11/5/2024	Daniel Machado	2.2
0.5	11/20/2024	Leah	4.2, 3.2, 5.3, 2.2
0.6	11/21/2024	Muhammed-Sharif	Revisions to 1.1, 2.3, 3.1, 4.1-4
1.0	11/21/2024	Leah	Added diagram to 4.2, submission of first draft

Document Control	ii
Distribution List.....	ii
Change Summary	ii
1. Introduction	1
1.1. Purpose and Scope	1
1.2. Project Executive Summary.....	1
1.2.1. System Overview	1
1.2.2. Design Constraints.....	2
1.2.3. Future Contingencies	2
1.3. Document Organization	2
1.4. Project References	2
1.5. Definitions, Acronyms, and Abbreviations	3
1.5.1. Definitions This section lists terms used in this document and their associated definitions.....	3
Table 1: Term Definitions	3
1.5.2. Acronyms	3
1.5.3. Abbreviations	3
2. System Architecture.....	5
2.1. System Software Architecture.....	6
2.1.1. Network Generation	6
2.1.2. Visualization of Networks.....	6
2.2. Internal Communications Architecture.....	7
3. Human-Machine Interface	8
3.1. Inputs.....	8
3.2. Outputs.....	8
4. Detailed Design	10
4.1. Hardware Detailed Design.....	10
4.2. Software Detailed Design	10
4.2.1. Database Management Subsystem	11
4.3. Internal Communications Detailed Design	11
5. External Interfaces.....	12
5.1. Hardware Interface Architecture.....	12
5.2. Hardware Interface Detailed Design.....	12
5.3. Software Interface Architecture	12
5.4. Software Interface Detailed Design	12

6. System Integrity Controls	13
------------------------------------	----

1. Introduction

1.1. Purpose and Scope

This SDD aims to document the software design of the project “Large-Scale Design and Analysis of Neural Networks”. This system addresses a need for structured neural network optimization, and to do so it will incorporate tools for automatic statistical analysis, visualization, and weight distribution tracking across multiple NN architectures, all while providing insights into network convergence and optimizing architecture from a statistical perspective.

The key objectives of this system include:

- Automating statistical analysis for neural networks.
- Providing user-friendly tools for visualization and convergence tracking.
- Enabling flexible design, which supports multiple neural network architectures.

The application of this system would be beneficial to the goals of any field in neural networks science. This is especially regarding the long-term accuracy of the training that results from these neural networks and the much-needed organization required to make these pipelined neural networks more efficient and consistent in accuracy/convergence.

1.2. Project Executive Summary

This project follows SCRUM. We have a Scrum Master, and a Product Owner, John Thompson. The team meets biweekly to talk about the tasks that have to be done, as well as discuss any issues that anyone might be having. We meet once a week with the product owner to give updates on the product and receive feedback on any changes that have to be made, and make sure we are staying within the product vision.

1.2.1. System Overview

The system offers tools to automate tasks associated with neural network evaluation. Key subsystems include:

1. **Data Ingestion and Preprocessing:** Prepares input data for model training.
2. **Neural Network Training Module:** Configures and trains multiple architectures.
3. **Analysis and Visualization:** Provides statistical insights and graphical representations like loss graphs and histograms.

Figure 1. System context diagram

1.2.2. Design Constraints

The only constraints of the system is that the user must have the correct dependencies to work with PyTorch, which will all be included in this document. There is the possibility to train the networks in parallel, which will require a multi-cored system.

1.2.3. Future Contingencies

The exact architecture of the system is not set in stone, so it is open to change. For now, it will simply be a class which contains different methods the user can use to perform analysis and visualization on a given neural network. Future contingencies may include more Indepth analysis and methods for re-running a specific neural network with deliberate analysis

1.3. Document Organization

This document is organized into six sections: introduction, system architecture, human-machine interface, detailed design, external interfaces, and system integrity controls.

1.4. Project References

References:

What are Neural Networks: Hands-On Machine Learning with Scikit-Learn and TensorFlow

Bayesian Neural Networks: https://www.cs.toronto.edu/~duvenaud/distill_bayes_net/public/

Pytorch API references: <https://pytorch.org/docs/stable/index.html>

1.5. Definitions, Acronyms, and Abbreviations

1.5.1. **Definitions** This section lists terms used in this document and their associated definitions.

Table 1: Term Definitions

Term	Definition
Pytorch	A machine-learning library based on the open source library on Lua.
Baye's Theorem	A theorem on probability, or statistical inference, that prior probability can be combined with new values and evidence to create a posterior probability which is updated as it becomes available.
Bayesian Neural Networks	Neural Networks whose weight distributions are modeled after probabilities using Baye's Theorem.
Architecture	Structural and logical layout and overall organization of a Neural Network.
Bins	Grouping continuous data into categories or intervals to use data as if it were discrete.
Weights	The connections between nodes/units in a neural network.
Histogram	Visualizing the distribution of continuous data into a graph and grouping into bins, similar to a bar-graph.
Convergence	Refers to the point where a model/network [learning] reaches a stable state and changes to its performance are at a minimum.
Tensors	Multidimensional arrays to store data used by neural networks

1.5.2. Acronyms

<< *Provide a list of used acronyms and their associated definitions.* >> his section lists the acronyms used in this document and their associated definitions.

Table 2: Acronym Definitions

Term	Definition
SDD	System Design Document
NN	Neural Network
BNN	Bayesian Neural Network
BN	Batch Normalization
DL	Deep Learning
SRS	Software Requirements Specification
ML	Machine Learning
CONV	Convergence

1.5.3. Abbreviations

< his section lists the abbreviations used in this document and their associated definitions.

Table 3: Abbreviations Definitions

Term	Definition
e.g.	For example
Etc.	Ex cetera

2. System Architecture

The Architecture of our system goes as follows, utilizing a layered architecture, with Python-based modules handling network generation, visualization and statistical analysis. This class subsystem for the test engineer to enter test parameters, such as viewing a live or previously recorded mission profile

Data Ingestion and Preprocessing

Responsible for loading and preparing data (e.g., normalization, tagging).

Prepares data to ensure compatibility with neural network training modules.

Neural Network Training Module

Handles the setup, training, and testing of different neural network architectures.

Includes interfaces for modifying architecture (e.g., layers, activation functions).

Analysis and Visualization

Provides tools for analyzing weight distributions, activation functions, and performance metrics. This is where the binning / visualization methods will be.

Includes visualization tools to represent weight distributions and other results.

2.1. System Software Architecture

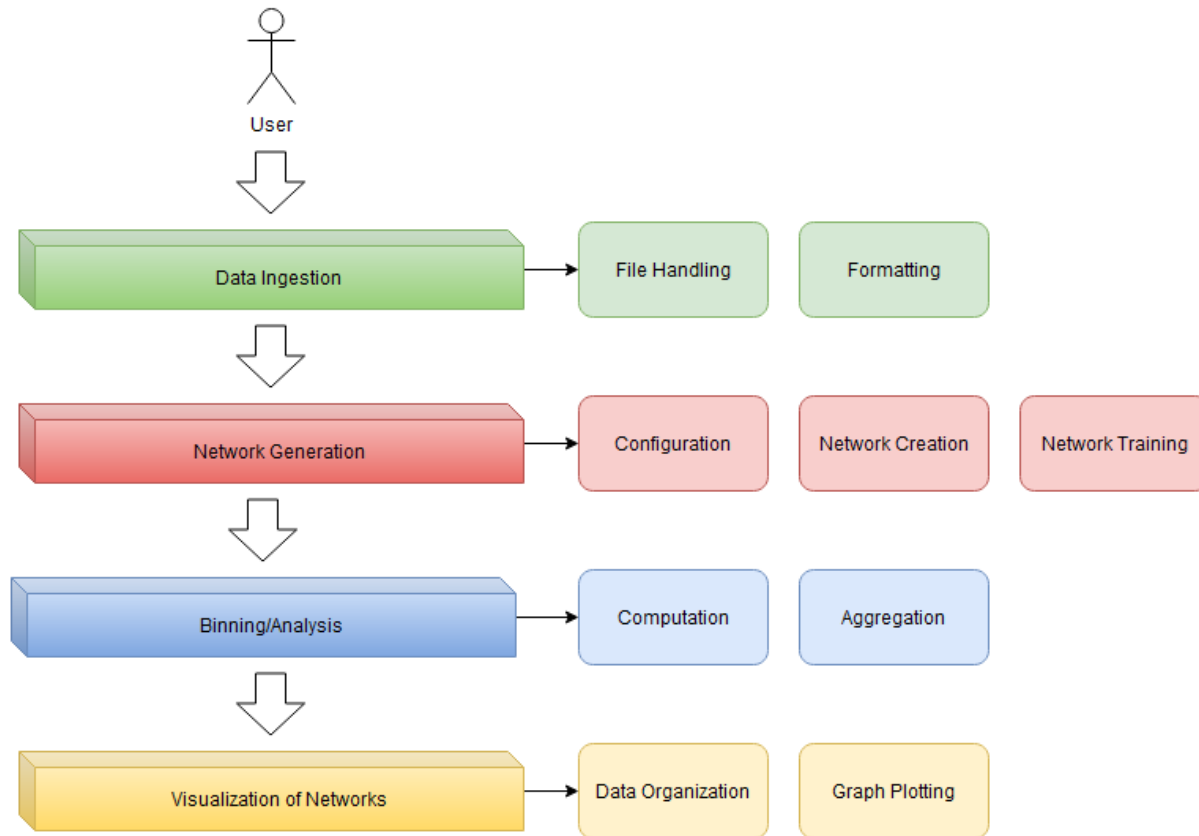


Figure 3. Software Architecture

The system is composed of a python package that runs based off a loose version of Pipeline and Layered Architecture. It is divided into different modules, each having different functionality within the layers of the pipeline. The modules that are in development consist of the following:

2.1.1. Network Generation

This module handles creation and training of a number of networks based on user parameters.

2.1.2. Visualization of Networks

This module allows the user to visualize network convergence and see if any changes to the network must be made before continuing to train the networks

2.2.3. Statistical Analysis of Weights

This module will be split into more modules. This module performs the different statistical analysis methods needed to understand the data.

2.2.4. Binning of Weights

This module takes care of the binning of the weights, and any processing regarding it. This is done before the statistical analysis, as the analysis is done on the weights after the binning.

2.2. Internal Communications Architecture

Modules communicate via Python function calls. No external communication protocols are required.

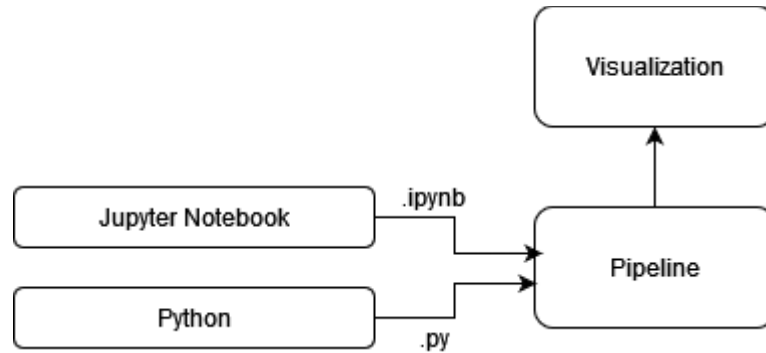


Figure: Function Calls Communications Diagram

3. Human-Machine Interface

3.1. Inputs

This system is a library made for specialized users. It is not meant to have a GUI, but rather user interact using the set API. The API so far is the following:

Constructor of network analyzer:

Network Architecture – The NN to perform analysis on.

Amount to produce – The number of networks to create.

Success criteria – Callback function that determines whether the training of a network has been successful or not.

Max Attempts – The maximum number of attempts to try and train a network.

Training the Network:

Training Loader – A Data Loader that contains the dataset. It is a Pytorch class.

Number of epochs – How many epochs to train for.

Loss Criterion- The method for which out losses are calculated for

3.2. Outputs

The System will display a variety of outputs including:

- *Graph Outputs/Visuals*
 - *Loss Graphs*
 - *Histograms Fitted to Weight Probability Distributions*
 - *Binning*
- *Reports/Metrics*
 - *Loss*
 - *Accuracy of model*
 - *Success Criteria (using loss graph tool)*

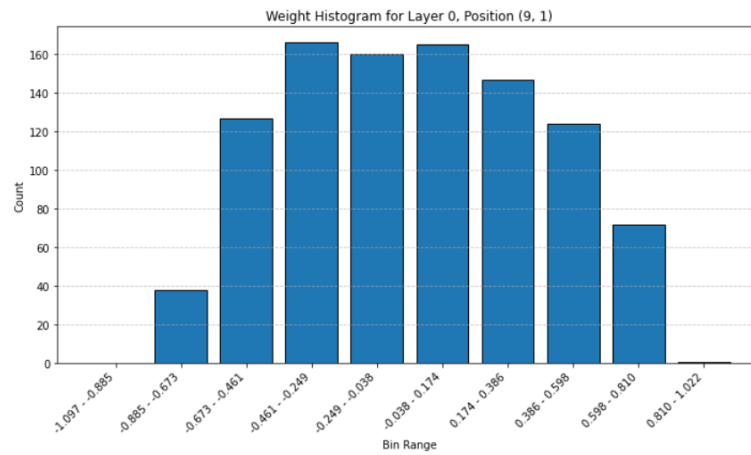


Figure 1: Histograms Fitted to Probability Distributions

4. Detailed Design

4.1. Hardware Detailed Design

N/A, system is entirely implemented in software.

4.2. Software Detailed Design

The software comprises:

1. **Data Ingestion Module:** Handles file-to-array conversions.
2. **Training Module:** Implements training pipelines.
3. **Visualization Module:** Generates graphs and statistical outputs

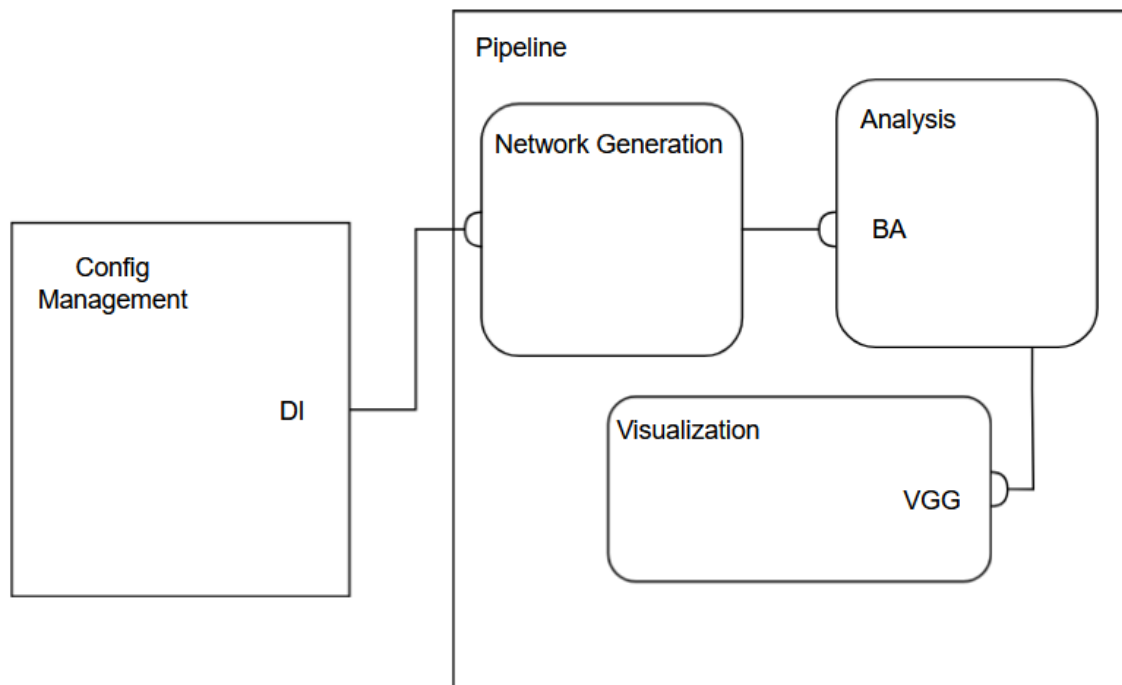


Figure 4. System collaboration graph

4.2.1. Database Management Subsystem

Contract <BA>: *Binning Algorithm*

Server: *Analysis*

Clients: *Data Ingestion Module*

Description: *Binning Algorithm receives epoch data from network and applies a specified binning strategy and returns binned dataset.*

Contract <NG>: *Network Generation*

Server: *Network Generation Module*

Clients: *NN Management*

Description: *Receives network architecture and dataset files to generate a neural network based off user selection of number of networks and hyperparameters for training.*

Contract <LGG>: *Loss Graph Generation*

Server: *Visualization Module*

Clients: *Analysis Module*

Description: *Takes trained neural networks and test dataset to compute loss and prepare for visualization in the form of a bar-graph.*

Contract <DI>: *Data Ingestion*

Server: *Data Ingestion Module*

Clients: *Pipeline*

Description: *reads user-provided files and provides them in a format compatible with network generation(e.g. NumPy arrays).*

Contract <V>: *Visualization*

Server: *Network Generation Module*

Clients: *Pipeline*

Description: *takes generated data from analysis and transforms it into a comprehensive diagram/visualization.*

Contract <A>: *Analysis*

Server: *Network Generation*

Clients: *Visualization Module*

Description: *Take generated data from network generation and analyzes it into a comprehensive dataset/output.*

4.3. Internal Communications Detailed Design

Modules interact through direct function calls and shared data structures like NumPy arrays and PyTorch tensors. Network Generation communicates and sends results to multiple modules, such as Analysis and Visualization.

5. External Interfaces

N/A – No external hardware interfaces.

5.1. Hardware Interface Architecture

N/A

5.2. Hardware Interface Detailed Design

N/A

5.3. Software Interface Architecture

Communication within the software occurs locally within any program running the Jupyter Notebook files. The pipeline communicates directly and internally between modules via function calls inside the python objects. No gateway or federation is present in the system.

5.4. Software Interface Detailed Design

Since the system operates locally and does not require any network-based communication protocols, all data exchange occurs internally between modules using Python function calls and in-memory objects, such as NumPy arrays and PyTorch tensors. This project requires data formatting in the process for Data Ingestion where files such as .csv must be formatted in Network Generation to perform as numpy arrays.

6. System Integrity Controls

This system does not currently process or store sensitive information, so no additional security measures are required.